

Compiladores 2026–2

Práctica 2

Análisis sintáctico descendente

Prof. Ariel Adara Mercado Martínez

Ayud. Carlos Gerardo Acosta Hernández

Adud. Luis Mario Escobar Rosales

Rodrigo André Decuir Fuentes

No. Cuenta: 320022711

ojo@ciencias.unam.mx

Fecha de entrega: 14 de abril



Ejercicios

Para la gramática $G = (N, \Sigma, P, S)$ descrita por las siguientes producciones:

```
P = {
  programa -> declaraciones sentencias
  declaraciones -> declaraciones declaracion | declaracion
  declaracion -> tipo lista-var ;
  tipo -> int | float
  lista_var -> lista_var, identificador | identificador
  sentencias -> sentencias sentencia | sentencia
  sentencia -> identificador = expresion; | if (expresion) sentencias else sentencias |
  while (expresion) sentencias
  expresion -> expresion + expresion | expresion - expresion | expresion * expresion |
  expresion \ expresion | identificador | numero
  expresion -> ( expresion )
}
```

1. Determinar en un archivo Readme, en formato Markdown (.md) o LaTeX (.tex) – con su respectivo PDF, para este último –, los conjuntos N, Σ y el símbolo inicial S . (0.5 pts.)

Respuesta: $N = \{\text{programa, declaraciones, sentencias, declaracion, tipo, lista_var, sentencia, expresion}\}$, $\Sigma = \{\text{identificador, int, float, if, (,), while, *, -, +, \, , numero, else, ;, ', ', =}\}$, $S = \{\text{programa}\}$.

2. Mostrar en el archivo el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario. (1 pts.)

Respuesta: Es necesario eliminar la ambigüedad porque no hay precedencia de operadores ni asociatividad. Lo podemos hacer transformando la producción "expresion" del siguiente modo (agregando las producciones termino y factor):

```
expresion -> expresion + termino | expresion - termino | termino
termino -> termino * factor | termino \ factor | factor
factor -> identificador | numero | (expresion)
```

3. Mostrar en el archivo el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario. (1 pts.)

Respuesta: si hay recursividad izquierda, por lo que hay que eliminarla:

Producciones	Reglas
declaraciones -> declaraciones declaracion declaracion declaraciones -> declaracion declaraciones' declaraciones' -> declaracion declaraciones' ϵ	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
lista_var -> lista_var, identificador identificador lista_var -> identificador lista_var' lista_var' -> , identificador lista_var' ϵ	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$

<pre> sentencias -> sentencias sentencia sentencia sentencias -> sentencia sentencias' sentencias' -> sentencia sentencias' ε </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
<pre> expresion -> expresion + termino expresion - termino termino expresion -> termino expresion' expresion' -> + termino expresion' - termino expresion' ε </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$
<pre> termino -> termino * factor termino \ factor factor termino -> factor termino' termino' -> * factor termino' \ factor termino' ε. </pre>	$A \rightarrow A\alpha \mid \beta$ $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$

4. Mostrar en el archivo el proceso de factorización izquierda o justificar, en caso de no ser necesario. (1 pts.)

Respuesta: No es necesario, ya que no hay producciones que compartan prefijos.

5. Mostrar en el archivo los nuevos conjuntos N y P . (0.5 pts.)

Respuesta:

$N = \{\text{programa, declaraciones, declaraciones', sentencias, sentencias', declaracion, tipo, lista_var, lista_var', sentencia, expresion, expresion', termino, termino', factor}\}.$

```

P = {
programa -> declaraciones sentencias
declaraciones -> declaracion declaraciones'
declaraciones' -> declaracion declaraciones' | ε
declaracion -> tipo lista-var ;
tipo -> int | float
lista_var -> identificador lista_var'
lista_var' -> , identificador lista_var' | ε
sentencias -> sentencia sentencias'
sentencias' -> sentencia sentencias' | ε
sentencia -> identificador = expresion;
| if (expresion) sentencias else sentencias
| while (expresión) sentencias
expresion -> termino expresion'
expresion' -> + termino expresion' | - termino expresion' | ε
termino -> factor termino'
termino' -> * factor termino' | \ factor termino' | ε
factor -> identificador | numero | (expresion)
}

```

6. Definir el Analizador Léxico (lexer.ll) con las acciones léxicas correspondientes. (2 pts.)

Respuesta: Listo. Ya se encuentra definido en su archivo correspondiente: src/lexer.ll

7. Implementar el Analizador Sintáctico (Parser.cpp) de descenso recursivo, documentando las funciones de cada No-Terminal, de forma que el programa descrito en el archivo de *prueba* sea reconocido y aceptado por el analizador resultante (4 pts.)

Respuesta: Se implementó el analizador en Parser.cpp siguiendo directamente la gramática transformada. Cada no-terminal tiene una función correspondiente:

- programa() — símbolo inicial; llama a declaraciones() y sentencias()
- declaraciones() / declaraciones_prima() — reconocen una o más declaraciones de variables
- declaracion() — reconoce tipo lista_var
- tipo() — reconoce int o float
- lista_var() / lista_var_prima() — reconocen una lista de identificadores separados por comas
- sentencias() / sentencias_prima() — reconocen una o más sentencias
- sentencia() — reconoce asignación (id = expr ;), if-else y while
- expresion() / expresion_prima() — reconocen sumas y restas con asociatividad izquierda
- termino() / termino_prima() — reconocen multiplicaciones y divisiones (mayor precedencia)
- factor() — reconoce identificador, número o expresión parentizada

La función eat() consume el token actual y solicita el siguiente al lexer. Ante un token inesperado se invoca error(), que reporta el problema y la línea. El análisis inicia en parse(), que también verifica que se haya consumido toda la entrada al terminar.

8. Documentar el código. (0.25 pts) - **No lo hice**
9. Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos. (0.25 pts) - **No lo hice**
10. Modificar el Makefile para poder simplificar el proceso de compilación de la práctica. (1 pts)
- **Si lo hice.**